

Royaume du Maroc



Ministère de l'Éducation Nationale,
du Préscolaire et des Sports

CLASSES PRÉPARATOIRES AUX GRANDES ÉCOLES

PROGRAMME ET INSTRUCTIONS OFFICIELLES

Filières MP / PSI

INFORMATIQUE

TABLE DES MATIÈRES

1	Préambule	2
2	Contexte de la nouvelle réforme de l'informatique en C.P.G.E.	3
3	Objectifs généraux de la formation	4
4	Organisation et recommandations pédagogiques	6
1 -	Organisation temporelle de la formation	6
2 -	Recommandations pédagogiques	8
5	Contenu du programme	10
1 -	Première période	10
1 - 1 -	La complexité algorithmique	10
1 - 2 -	Algorithmes de tri	11
1 - 3 -	Les arbres binaires	12
1 - 4 -	Algorithmes gloutons	13
1 - 5 -	Programmation dynamique	13
1 - 6 -	Meta Heuristique	14
2 -	Deuxième période	16
2 - 1 -	Introduction à la théorie des graphes	16
2 - 2 -	Introduction à l'intelligence artificielle	18
2 - 3 -	Introduction à la théorie des jeux	19
2 - 4 -	Les bases de données relationnelles	20

1

PRÉAMBULE

Étant donné que les technologies de l'information et les sciences du numérique évoluent en permanence, le programme d'enseignement d'informatique en deuxième année MP et PSI doit également être constamment adapté pour suivre cette évolution.

En conséquence de cette nécessité, le Ministère de l'Éducation Nationale, du Préscolaire et des Sports, déploie d'importants efforts pour réviser régulièrement les programmes d'informatique dans les classes préparatoires aux grandes écoles (CPGE) au Maroc. Par conséquent, ce document a été élaboré dans le but de :

- Définir la nature et les caractéristiques de l'informatique en tant que discipline d'enseignement dans les classes préparatoires des grandes écoles (CPGE).
- Délimiter le cadre et la vision du programme d'informatique en CPGE.
- Indiquer les compétences à développer chez les élèves.
- Fixer les finalités et les objectifs de chaque partie du programme.
- Établir une approche pédagogique qui servira de guide pour la préparation des activités d'apprentissage en informatique.
- Présenter le programme ainsi que la progression qui lui est attachée.
- Conseiller des exercices et des exemples d'applications relatifs aux différents éléments de ce programme.

Le but de ce document est de garantir que l'enseignement de l'informatique dans les classes préparatoires est en adéquation avec les réalités technologiques actuelles afin de préparer les élèves à réussir dans le monde en constante évolution de la technologie de l'information.

2

CONTEXTE DE LA NOUVELLE RÉFORME DE L'INFORMATIQUE EN C.P.G.E.

La nouvelle réforme du programme informatique dans les classes préparatoires vise à enrichir le contenu proposé aux élèves afin de leur fournir des outils plus novateurs et actuels qui peuvent leur être utiles dans la suite de leur parcours dans les grandes écoles d'ingénierie, ainsi que dans leur vie professionnelle.

Cette réforme vise également à promouvoir une approche interdisciplinaire, où l'informatique est intégrée à d'autres domaines tels que la physique, les mathématiques et les sciences d'ingénieur afin de permettre aux élèves d'utiliser l'informatique aussi bien telle quelle ou bien comme un outil pour résoudre des problèmes réels.

Il convient cependant de souligner que cette réforme ne se limite pas simplement à une extension du programme existant ; elle s'inscrit dans une démarche d'amélioration continue, visant à enrichir l'expérience des élèves en introduisant de nouveaux chapitres et en actualisant les contenus existants.

Parmi les nouveautés de cette réforme, l'intégration de quelques techniques de résolution de problèmes telles que l'approche gloutonne, la programmation dynamique, ainsi que les méthodes métaheuristiques représentées par la méthode de recuit simulé. Ces outils vont permettre d'élargir le champ des choix pour les élèves, et leur fournir une base solide pour résoudre des problèmes en relation avec leur domaine. De plus, ils pourront les utiliser pour rechercher des solutions à certains problèmes de la vie réelle à travers leurs travaux de TIPE.

3

OBJECTIFS GÉNÉRAUX DE LA FORMATION

Ce manuel a pour objectif de présenter le contenu de la nouvelle réforme et mettre à jour la liste des objectifs visés en tenant en compte les nouvelles parties introduites.

Les objectifs globaux du programme informatique peuvent être énumérés comme suivant :

- **Utiliser l'informatique pour modéliser et résoudre un problème de la vie réelle** Les élèves doivent être en mesure d'utiliser les différentes techniques et structures de données disponibles pour analyser, modéliser et résoudre efficacement des problèmes concrets. En comprenant les principes sous-jacents des algorithmes et des structures de données, les élèves peuvent choisir les approches les plus adaptées à leurs problèmes spécifiques. Ils doivent également être capables de concevoir des solutions informatiques robustes qui tiennent compte des contraintes de performance, de mémoire et de temps d'exécution.
- **Structurer une solution informatique** : La structuration d'une solution pour un problème informatique est une étape primordiale pour concevoir des solutions modulaires et faciles à entretenir. En décomposant les problèmes complexes en sous-problèmes plus gérables, les élèves peuvent mieux gérer la complexité et améliorer la lisibilité de leur code. En outre, cette approche favorise la réutilisation du code et facilite la détection et la correction des erreurs. Les élèves sont incités à décomposer leurs solutions en petites fonctions qui seront par la suite regroupées pour fournir la solution globale. Ce processus permet de mieux comprendre et organiser leurs codes, favorisant ainsi leurs maintenances.

➡ **Valider une solution** : cela nécessite de justifier que la solution fournit exactement le résultat attendu et que son temps d'exécution est raisonnable. Pour ce faire, l'élève doit développer un esprit critique envers les différents programmes qui lui seront demandés à réaliser. Cet esprit critique est très favorable pour familiariser les élèves à corriger et à améliorer leurs réalisations. En examinant de manière critique leurs propres solutions, les élèves peuvent identifier les erreurs potentielles, les inefficacités et les améliorations possibles. Cela favorise un processus d'apprentissage actif où les élèves acquièrent une compréhension approfondie des algorithmes, des structures de données et des bonnes pratiques de programmation. De plus, cette compétence d'évaluation critique est transférable à d'autres domaines de leur vie professionnelle, où la capacité à évaluer et à améliorer les solutions est essentielle.

4

ORGANISATION ET RECOMMANDATIONS PÉDAGOGIQUES

1 - Organisation temporelle de la formation

Le programme d'informatique en 2ème année des classes préparatoires vient consolider celui déjà vu en première année. Le programme se porte toujours sur les notions de base de la programmation en utilisant le langage Python. Le contenu de la formation en 2ème année est décomposé en deux périodes.

Programme de la première période :

Pendant la première période, la notion de complexité algorithmique est abordée. Les élèves sont invités à calculer la complexité de leurs programmes, tant dans le scénario du pire cas que dans celui du meilleur cas, afin d'évaluer leurs programmes et de vérifier leur faisabilité. Comprendre la complexité algorithmique est crucial car elle permet d'estimer les ressources nécessaires à l'exécution d'un algorithme en fonction de la taille de l'entrée, ce qui est essentiel pour évaluer les performances et anticiper les limites de nos programmes.

Le concept de tri est également exploré. Cette notion est déjà introduite en première année, où les élèves sont déjà initiés à des algorithmes de tri simples tels que le tri par sélection, le tri par insertion et le tri à Bulles. En deuxième année, le cursus approfondit ces connaissances en présentant aux élèves des méthodes de tri plus efficaces, telles que le tri rapide, le tri par fusion. Cela permet aux élèves de développer une compréhension plus

nuancée des algorithmes de tri et de choisir les approches les plus adaptées en fonction des besoins spécifiques de leurs projets.

Au cours de cette première période, les élèves exploreront une structure de données bien connue sous le nom d'arbre. Cette structure permet de représenter des données hiérarchiques, facilitant ainsi leur manipulation. Les arbres sont utilisés dans de nombreux domaines, tels que l'informatique, la biologie et la linguistique. En prenant des exemples de ces domaines, Cela permettra aux élèves de mieux comprendre leur utilité et d'exploiter leur compréhension pour créer et manipuler cette structure pour résoudre certains problèmes en relation avec leur domaine.

La première partie inclut également de nouveaux chapitres introduisant les méthodes de résolution de problèmes d'optimisation. Il s'agit notamment de l'approche gloutonne, de la programmation dynamique et des métaheuristiques. Ces techniques sont fondamentales pour résoudre un large éventail de problèmes d'optimisation rencontrés dans divers domaines, tels que l'ingénierie, les sciences économiques, la logistique, etc. Les élèves apprennent à formuler des problèmes d'optimisation, à concevoir des solutions efficaces et à évaluer les compromis entre différentes approches algorithmiques.

Programme de la deuxième période :

Pendant cette deuxième période, les élèves vont explorer le domaine d'intelligence artificielle, et plus précisément le sous-domaine de l'apprentissage automatique (Machine Learning). Ils seront amenés à manipuler l'algorithme d'apprentissage supervisé K plus proches voisins (KNN) pour réaliser des classifications ou des régressions. De plus, ils vont pouvoir expérimenter le domaine de l'apprentissage non supervisé à travers l'algorithme K-means utilisé pour la segmentation de données. Ces notions visent à initier les élèves au domaine de l'intelligence artificielle et à leur permettre d'acquérir une vision plus claire et technique de ce domaine. Cela devrait leur permettre d'intégrer plus facilement cet outil pour résoudre des problèmes liés à leur Travaux d'Initiative Personnelle Encadrée (TIPE) ou à d'autres disciplines.

Pendant cette deuxième période, les élèves découvriront une nouvelle structure de données : Les graphes. Après avoir exploré la terminologie associée à cette structure, ils seront amenés à s'attaquer à des problèmes classiques dont la résolution nécessite l'utilisation de ces structures. Le problème de recherche du chemin le plus court à origine unique est un bon point de départ. L'algorithme de Dijkstra pourra être utilisé pour résoudre ce type de problème.

Le Troisième chapitre de cette partie introduira aux élèves la notion des jeux à somme nulle, un concept fondamental de la théorie des jeux. Cette notion s'inscrit dans le cadre du chapitre sur les algorithmes heuristiques, où les élèves vont apprendre à utiliser ces algorithmes pour analyser et implémenter des jeux à somme nulle. L'algorithme Minimax fait partie du programme.

En dernière partie, les élèves exploreront le domaine des bases de données pour manipuler l'algèbre relationnelle et le langage SQL (Structured Query Language) afin d'organiser et gérer des données. En combinant la théorie de l'algèbre relationnelle avec la pratique de SQL, les élèves acquerront les compétences nécessaires pour concevoir, manipuler et interroger des bases de données relationnelles, ce qui est essentiel dans de nombreux domaines de l'informatique et de l'ingénierie.

Dans la suite de ce document, nous présenterons les détails des différents chapitres de cette deuxième année

2 - Recommandations pédagogiques

L'enseignement de l'informatique en deuxième année MP et PSI dans les classes préparatoires aux grandes écoles vise plusieurs objectifs permettant aux élèves d'appliquer les bases de programmation acquises en première année pour résoudre numériquement des problèmes d'ordre scientifiques.

Le programme de l'informatique est organisé en deux périodes de volume sensiblement équivalent. Ce découpage en deux périodes d'enseignement doit être respecté, En revanche,

au sein de chaque période, aucun ordre particulier n'est imposé et chaque professeur conduit en toute liberté l'organisation de son enseignement.

Dans le cadre de ce programme, plusieurs recommandations pédagogiques sont à prendre en compte :

- ➡ Raisonnement algorithmique privilégié : Il est recommandé de mettre l'accent sur le raisonnement algorithmique beaucoup plus que sur la syntaxe du langage de programmation.
- ➡ Exemples et exercices multidisciplinaires : Il est recommandé, de donner des exemples et de proposer des exercices inspirés des autres disciplines (mathématiques, physique, chimie et sciences d'ingénieur), concrets et inspirés du monde réel.
- ➡ Flexibilité dans l'organisation de l'enseignement : Offrir une flexibilité dans l'organisation de l'enseignement, laissant aux professeurs la liberté d'adapter leur approche.
- ➡ Éviter les développements formels trop théoriques : Les développements formels ou trop théoriques doivent être évités. Ils ne correspondent pas au cœur de la formation de ces classes préparatoires.

5

CONTENU DU PROGRAMME

1 - Première période

1 - 1 - La complexité algorithmique

L'objectif de cette section est d'introduire la notion de complexité algorithmique et d'encourager les élèves à l'utiliser comme une métrique permettant d'évaluer un programme informatique et de confirmer sa faisabilité. Les élèves doivent être capables de calculer la complexité pour un programme itératif ou récursif.

Introduction à la complexité

Introduire la notion de complexité et expliquer son intérêt.

Expliquer la différence entre la complexité temporelle et la complexité en espace.

Enumérer les différents cas d'étude de la complexité (meilleur, moyen, pire). La notion de complexité est abordée en classe prépas uniquement pour les scénarios meilleur et pire des cas.

Compter le nombre d'opérations

Calculer le nombre d'opérations élémentaires dans un programme itérative.

Notation asymptotique	Introduire la notation grand O et expliquer son intérêt.
Calculer la complexité pour un programme	Calculer la valeur asymptotique pour un programme itératif. Enumérer les ordres asymptotiques classiques Calculer l'ordre asymptotique pour un programme récursif (on peut passer par des approximations pour réduire les calculs)
Calculer la complexité spatiale d'un programme	Se limiter au calcul asymptotique dans le meilleur et le pire des cas

1 - 2 - Algorithmes de tri

Cette section fait suite à celle étudiée en première année. L'objectif est d'explorer des algorithmes de tri plus efficaces tel que le tri rapide et le tri par fusion. Il est particulièrement intéressant de mettre l'accent sur certaines caractéristiques des algorithmes de tri, telles que la notion de tri en place et de stabilité.

Tri rapide	Définir le principe de l'algorithme de tri rapide. Implémenter le principe de cet algorithme. Comparer sa complexité avec celles des autres algorithmes. Discuter le choix du pivot.
Tri par fusion	Définir le principe de la méthode tri par fusion. Implémenter le principe de cette méthode. Comparer sa complexité avec celles des autres méthodes.

1 - 3 - Les arbres binaires

Étant une structure de données très intéressante, les arbres présentent un moyen efficace pour stocker des informations et faciliter leur utilisation. Les élèves sont incités à manipuler cette structure. Ils doivent être en mesure de la définir en utilisant des listes, de la parcourir en largeur et en profondeur, de calculer sa hauteur, etc. Il est également intéressant de manipuler quelques structures d'arbre telles que les arbres binaires de recherche, les tas et autres.

Terminologie de base : noeud, racine, feuille, noeud interne, degré, ordre, chemin, profondeur, hauteur	Définir les différents composants d'un arbre ainsi que ses caractéristiques. Calculer la hauteur d'un arbre. Calculer la profondeur d'un nœud
Représenter et implémenter la structure arbre binaire	Définir un arbre en utilisant une liste
Parcourir un arbre binaire	Parcourir un arbre en profondeur en utilisant les parcours préfixe, infixe et postfixe. Parcourir également un arbre en largeur
Implémentations et manipulations d'autres variantes	Manipuler et implémenter des structures d'arbres classiques tel que les arbres binaires de recherche et les tas.
Manipuler et implémenter les arbres binaires de recherche	Relation d'insertion dans un arbre binaire de recherche, recherche d'une valeur, hauteur, Insertion, création, etc.

1 - 4 - Algorithmes gloutons

L'intérêt de cette section est d'appliquer la technique de l'approche gloutonne pour résoudre des problèmes d'optimisation. Il est intéressant d'analyser la structure d'une solution gloutonne et de montrer, à l'aide d'exemples concrets, que cette stratégie ne permet pas toujours d'atteindre une solution optimale. La nature gourmande de l'approche gloutonne se manifeste par ses choix localement optimaux sans une évaluation exhaustive des conséquences globales.

Expliquer de façon simple la stratégie gloutonne : la propriété de sous-structure optimale	Utiliser des exemples simples pour appliquer le principe de la stratégie gloutonne.
--	---

Contre-exemple de la non-optimalité.	Montrer à l'aide d'exemples faites à la main que la stratégie gloutonne ne garantit pas toujours l'aboutissement à la solution optimale
--------------------------------------	---

Résoudre des problèmes d'optimisation classiques par l'approche gloutonne	Exemples : Rendu de monnaie, sac à dos, arbre de huffman
---	--

1 - 5 - Programmation dynamique

L'objectif est de montrer aux élèves l'intérêt de la programmation dynamique et comment l'utiliser pour résoudre un problème d'optimisation. Les élèves doivent appliquer cette technique dans ses deux versions : Top-down et Bottom-up. Pour maîtriser cette technique, on peut aborder des exemples de problèmes à une ou deux dimensions.

Expliquer l'intérêt de la programmation dynamique	Expliquer à quoi sert la programmation dynamique et son utilité pour réduire la complexité temporelle de certains problèmes. Mettre en rapport le statut de la propriété de sous-structure optimale en programmation dynamique avec sa situation en stratégie gloutonne
Expliquer et implémenter les deux approches de la programmation dynamique (Top-Down et Bottom-Up)	Énumérer les caractéristiques de chaque méthode. Analyser un programme d'optimisation simple et extraire les formules récursives qui le décrivent. Utiliser ces formules récursives pour implémenter les deux approches de la programmation dynamique.
Implémenter des exemples classiques sur la programmation dynamique	Exemples : Plus longue sous séquence commune, distance de Levenshtein, sac à dos binaire, etc.

1 - 6 - Meta Heuristique

L'objectif de cette présentation est d'introduire l'approche métaheuristique dans le contexte de la résolution de problèmes d'optimisation. Les élèves doivent acquérir la compétence nécessaire pour appliquer cette approche à des problèmes relevant du domaine de l'informatique ou d'autres domaines en lien avec leur formation. Il est essentiel de souligner que l'approche métaheuristique ne garantit pas la découverte de la solution optimale, mais elle offre la capacité de trouver une solution approximative.

Définition et principe

Définir la notion de méta-heuristique et expliquer son principe.

Indiquer que ces méthodes sont employées pour aborder des problématiques variées issues de divers domaines.

Algorithme Recuit simulé

Expliquer son intérêt et son principe.

Implémenter son algorithme.

2 - Deuxième période

2 - 1 - Introduction à la théorie des graphes

L'objectif de cette section est de définir la notion de graphes, comme étant des structures de données, ainsi que leurs représentations en Python et leurs manipulations. À la fin de cette section, les élèves doivent être en mesure de concevoir des solutions en utilisant des programmes Python pour résoudre divers problèmes inspirés du monde réel, qui peuvent être représentés par des graphes.

Vocabulaire des graphes

Présenter les notions suivantes à travers des exemples : graphe orienté/non orienté, sommets et arcs/arêtes, ordre et degré (entrant et sortant), chemin/chaine et circuit/cycle, connexité dans les graphes non orientés.

Se limiter sur le cas des graphes simples : on n'évoque ni boucles, ni multi-arêtes.

Implémentation et manipulation des graphes

Expliquer comment implémenter des graphes à l'aide de listes d'adjacences (rassemblées dans une liste ou dans un dictionnaire) et à l'aide de matrice d'adjacence.

Donner des exemples pratiques de manipulation des graphes afin de bien maîtriser son implémentation. Par exemple :

- ➡ Ajout / suppression / test d'existence d'un arc/arête.
- ➡ Construction de la liste des voisins d'un sommet à partir de la matrice d'adjacence. etc.

Parcours d'un graphe :	Souligner les problèmes d'efficacité posés par l'implémentation des files par des listes et l'avantage d'utiliser la fonction <code>deque()</code> du module <code>collections</code> . Expliquer et implémenter sur des exemples simples le principe des algorithmes de parcours. Utiliser ces algorithmes de parcours pour détecter la présence d'un cycle/circuit dans un graphe, ou bien pour vérifier la connexité d'un graphe non orienté.
➡ Parcours en largeur et en profondeur.	
➡ applications.	
Pondération d'un graphe. Étiquettes des arcs ou des arêtes d'un graphe.	Expliquer l'intérêt d'ajout d'étiquettes aux arcs / arêtes d'un graphe à travers des exemples concrets.
Recherche d'un plus court chemin dans un graphe pondéré positivement	Expliquer le principe de l'algorithme de dijkstra à travers un exemple de graphe pondéré positivement. Introduire l'algorithme A^* comme étant une variante heuristique de Dijkstra.
Applications	Etudier quelques algorithmes classiques sur les graphes, comme : ➡ La coloration des graphes. ➡ distances dans un graphe (Floyd-Warshall), etc.

2 - 2 - Introduction à l'intelligence artificielle

L'objectif de cette section est d'initier les élèves, d'une manière générale, au domaine de l'intelligence artificielle, et en particulier à l'apprentissage automatique et qui est utilisé pour résoudre des problèmes de classifications et de segmentations. A la fin de cette section, les élèves doivent être capables de comprendre et de résoudre des problèmes en appliquant des algorithmes de l'intelligence artificielle.

Introduction à l'intelligence artificielle : Définir et expliquer son intérêt.

Énumérer ses sous-domaines.

Donner des exemples de son utilisation dans la vie courante.

Apprentissage supervisé :

➡ Problèmes de classification.

➡ Algorithme KNN.

➡ Matrice de confusion.

S'intéresser aux problèmes de classification auxquels on prédit la classe des nouvelles données, en se basant sur un ensemble de données déjà étiquetés avec leurs classes.

Présenter l'algorithme des k plus proches voisins (KNN) avec distance euclidienne comme exemple simple d'algorithme de classification, et l'appliquer sur un problème simple.

Expliquer comment évaluer la pertinence du choix du paramètre k de l'algorithme KNN à l'aide de la matrice de confusion.

Apprentissage non-supervisé :

➡ Problèmes de segmentation.

➡ Algorithme de k-moyennes.

S'intéresser aux problèmes de segmentations qui consistent à regrouper un ensemble d'éléments hétérogènes sous forme de sous groupes liés par des caractéristiques communes. Présenter l'algorithme de k-moyennes comme exemple simple d'algorithme de segmentation (clustering). Appliquer cet algorithme sur un exemple simple, et observez sa convergence vers des minimums locaux.

Applications

Appliquer ces algorithmes sur quelques problèmes.

2 - 3 - Introduction à la théorie des jeux

L'objectif de cette section est de fournir une introduction sur la théorie de jeux, en mettant l'accent sur le lien avec la théorie des graphes, et en se limitant seulement aux jeux d'accessibilité à deux joueurs. À la fin de cette section, les élèves devront avoir une compréhension solide des notions liés à la théorie des jeux d'accessibilité, ainsi que des idées sur les stratégies (algorithmes) utilisées pour les résoudre.

Jeux d'accessibilité à deux joueurs sur un graphe

Considérer des jeux d'accessibilité à deux joueurs modélisés par des graphes bipartis.

Expliquer les notions suivantes à travers un exemple de jeu d'accessibilité simple : état initial, états gagnants, partie du jeu, et graphe biparti du jeu.

Détermination des positions gagnantes par le calcul des attracteurs. Construction de stratégies gagnantes

Déterminer les positions gagnantes de chacun des joueurs en utilisant l'attracteur.

Expliquer comment construire les stratégies gagnantes pour chaque joueur (définie sur chaque position gagnante que le joueur contrôle).

Algorithme Minimax avec une heuristique	Introduire l'algorithme min-max comme alternatif dans le cas d'un jeu complexe ou le graphe associé est très gros. Utiliser une heuristique pour privilégier le choix de certaines positions durant le jeu. Appliquer l'algorithme Minimax sur un exemple de jeu simple.
---	---

2 - 4 - Les bases de données relationnelles

Cette partie vise à initier les élèves aux bases de données relationnelles et leur manipulation à l'aide du langage SQL qu'on utilise avec SQLite, un système de gestion de base de données léger . À la fin de cette section, les élèves devront être capables d'exprimer des requêtes SQL permettant d'insérer, mettre à jour et supprimer des données d'une base de données relationnelles, ainsi que d'interroger une base de données pour extraire des informations précises à l'aide de requêtes avancées.

Vocabulaire des bases de données relationnelles	Présenter les concepts suivants à travers quelques exemples simples : table (relation), attribut (colonne), enregistrement (ligne), domaine et types de données, schéma de tables.
Modèle entités et associations	Expliquer les notions suivantes à travers des exemples simples : entité, association, attributs, identifiant, et cardinalités.

Notion de clé :	Une clé primaire n'est pas forcément associée à une unique colonne même si c'est le cas le plus fréquent.
➡ Clé primaire.	
➡ Clé étrangère.	Expliquer le rôle des clés primaires et étrangères pour mettre le lien entre les différentes tables d'une BD relationnelle.
Passage du modèle entité-association au modèle relationnel : Transformation des entités et associations en des tables (relations).	Transformer une entité en une relation (table). Séparer une association * - * en deux associations 1 - *. Utiliser les clés primaires et clés étrangères pour traduire les associations 1 - 1 et 1 - *.
Langage SQL et Manipulation des données (insertion, modification et suppression des lignes d'une table).	Introduire le langage SQL et présenter SQLite comme SGBD à utiliser. Présenter la notion de requête SQL et donner la syntaxe des différentes requêtes de manipulation de données : requêtes INSERT INTO, UPDATE et DELETE FROM.
Interrogation d'une base de données relationnelle : Requête SELECT (projection) avec simple clause WHERE (sélection).	Expliquer les notions de projection et sélection, en donnant leurs notations en algèbre relationnel ainsi que les requêtes SELECT équivalentes. Expliquer l'intérêt d'utilisation du mot clé DISTINCT. Utiliser les opérateurs suivants pour exprimer des conditions : =, !=, >, <, >=, <=, AND, OR, NOT, IN, LIKE, BETWEEN ... AND ..., ...

Opérateurs ensemblistes : union, intersection, différence, et produit cartésien.	Donner des exemples de requêtes SQL auxquelles on combine les résultats de plusieurs requêtes avec UNION, INTERSECT ou EXCEPT. Expliquer avec un exemple la notion de produit cartésien, comme étant un outil permettant de combiner les résultats venant de différentes tables.
Jointure interne avec opérateur d'égalité (JOIN ... ON ...=...).	Introduire la notion de jointure en mentionnant le fait que le résultat d'un produit cartésien peut ne pas avoir de sens. Utiliser les jointures internes pour effectuer des requêtes croisées dans une base de données constituée de plusieurs tables. Consolider les acquis à travers des exemples d'application.
Renommage AS des tables et des colonnes	Expliquer, avec des exemples, l'intérêt du renommage temporairement d'une colonne ou d'une table dans une requête SQL.
Fonctions d'agrégation et groupement des lignes avec la clause GROUP BY	Appliquer des fonctions d'agrégation sur des groupes de lignes d'une table, qu'on partitionne selon un critère donné portant sur des colonnes (GROUP BY). Mettre en œuvre des agrégats, en utilisant les fonctions MIN, MAX, SUM, AVG et COUNT. Mentionner le fait que les fonctions d'agrégations peuvent s'appliquer sur un seul groupe qui contient toutes les lignes de la table (s'il n'y a pas de GROUP BY).

Filtrage des agrégats avec HAVING	Filtrer des agrégats avec HAVING pour limiter l'affichage sur les groupes qui vérifie une condition donnée. Marquer la différence entre WHERE et HAVING sur des exemples.
Modification d'affichage : Tri des résultats et limitation d'affichage	Utiliser le mot-clé ORDER BY pour trier les lignes résultats d'une requête SELECT, par rapport à la valeur d'une ou plusieurs colonnes. Utiliser le mot-clé LIMIT pour limiter le nombre de lignes affichées par une requête SELECT. Utiliser le mot-clé OFFSET pour sauter un certain nombre de lignes.
Requêtes imbriquées : Syntaxe et opérateurs de comparaison	Expliquer la syntaxe générale des sous-requêtes dans une clause WHERE, FROM, ou HAVING. Utiliser les opérateurs =, <, >, !=, <=, >=, IN / NOT IN, et EXISTS/NOT EXISTS, pour comparer les résultats de sous-requêtes. Donner quelques exemples de requêtes imbriquées.